Anantha Lakshmi
INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)

ACADEMIC PROJECT PRESENTATION · 2026–27

StudyMapper

AI-driven personalised learning and academic management — plan, learn, practise and improve in one workspace.

TEAM

P. Venubhaskar · T. Vasanth · V. Adarsh Kumar · B. Sai Vikas · C. Narendra

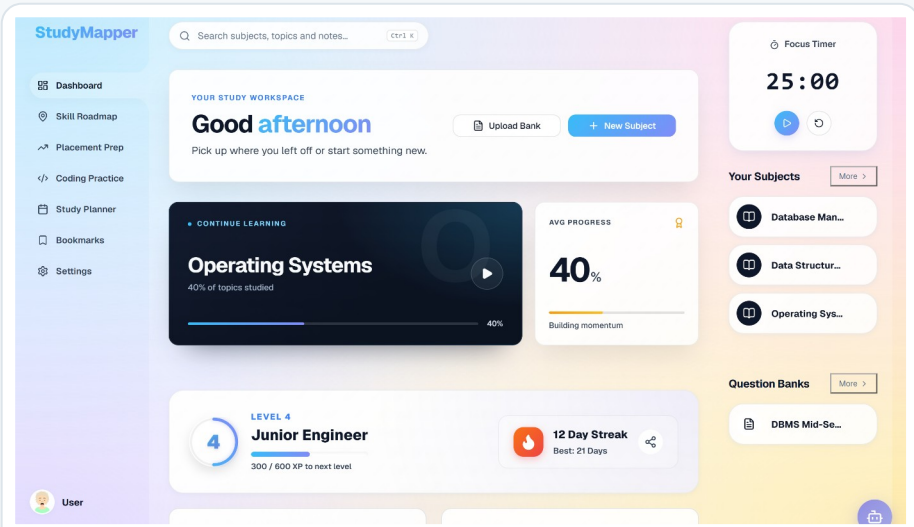
GUIDE · Ms. C. Sravani

Department of Computer Science & Engineering
Anantha Lakshmi Institute of Technology & Sciences (Autonomous)

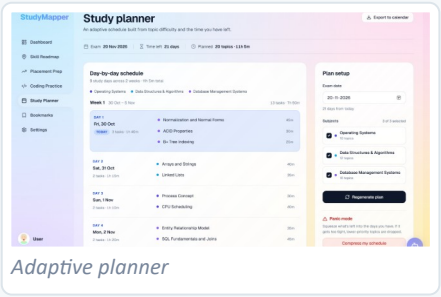
Live demo · study-mapper.vercel.app

47 / 47 tests

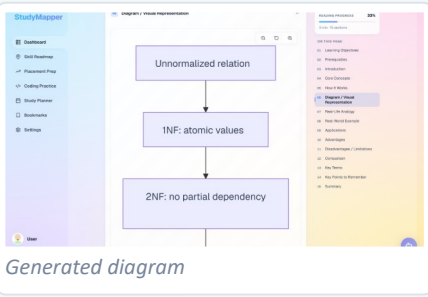
Everything shown in this deck is drawn from the project repository and the running application.



Dashboard — the daily workspace



Adaptive planner



Generated diagram

Genuine captures from the application at 1440 × 900, light theme. No mockups.

AGENDA

Eighteen slides, five acts

Why the system exists, what it does, how it is built, what is verified, and where it goes next.

01 FOUNDATIONS → **02 THE SYSTEM** → **03 ENGINEERING** → **04 EVIDENCE** → **05 CLOSE**

SLIDES 03–06

Abstract, problem, objectives and existing work.

- Context & motivation
- Problem definition
- Objectives & scope
- Literature review

OUTCOME

You will know the problem, the scope and the gap.

SLIDES 07–09

The product loop, its architecture and the method.

- Proposed modules
- Layered architecture
- Development method

OUTCOME

You will see how the nine modules connect.

SLIDES 10–12

The decisions that make it work in a browser.

- Technology choices
- Data model & flow
- Interface design

OUTCOME

You will see the data model and interface decisions.

SLIDES 13–14

What was tested, and what is not claimed.

- Verified results
- Contributions
- Limitations

OUTCOME

You will know exactly what was tested — and what was not.

SLIDES 15–18

Conclusion, next steps and sources.

- Conclusion
- Future scope
- References
- Questions

OUTCOME

You will know the next steps and the sources.

HOW TO READ THIS DECK

Teal marks the system, amber marks evidence, rose marks a limitation. Implementation claims come from the repository; every screenshot is from the running app.

A study loop that responds to the learner

StudyMapper turns a syllabus into an adaptive plan, generates study material, and measures what actually needs revision — in one browser-based workspace.

IN ONE PARAGRAPH

The system combines curriculum management, AI-assisted content generation, spaced-repetition scheduling and practice analytics. A student builds or imports a syllabus, receives a difficulty-aware study plan, studies generated notes and diagrams, practises with question banks, and sees weak topics recomputed from recent activity. Data lives locally in the browser; AI and sign-in are optional services.

WHY IT MATTERED

- **Study time is fixed; attention is not.**
  A plan that ignores difficulty and weak topics wastes the hours a student does have.

- **Feedback arrives after the exam.**
  Continuous practice data makes weakness visible while it can still be acted on.

- **Study tools are scattered.**
  Notes, plans, tests and progress sit in different apps, so context is lost between them.

DESIGN PRINCIPLE · The plan is an output of evidence, not a document.

Built as a single-page React application. No server is required for the core study loop.

THE LEARNING LOOP



Each pass writes back to the same store, so the next plan starts from evidence.

The gap is not content — it is response

Students rarely fail for lack of material. They fail because nothing in their day responds to what they actually understand.

01

Fragmented workflow

Notes live in one app, deadlines in another, tests in a third. Context is lost every time a student switches.

A student rewrites the same notes in three tools.

COST TO THE STUDENT

Revision restarts from zero.

02

Static study plans

A plan made once ignores topic difficulty, weak areas and slipping dates. It stays comfortable while the exam gets closer.

The weakest topic gets the same 30 minutes as the strongest.

COST TO THE STUDENT

Time goes where it is easiest, not where it is needed.

03

Feedback after the fact

Progress becomes visible only when marks arrive. Misconceptions survive weeks of practice unchallenged.

A wrong method is repeated until the mock exam.

COST TO THE STUDENT

The same mistakes repeat.

DESIGN IMPLICATION

Three failures define the requirements.

One workspace
context survives

→ **A plan that recomputes**
from real evidence

→ **Measurement inside practice**
not after it

Four objectives, each tied to something observable

Scope was deliberately narrow: the individual study loop for a CSE undergraduate — not institutional administration.

01

Make planning adaptive

Regenerate the schedule from exam date, topic difficulty and weak scores.

Measured by · A plan that visibly changes after practice.

02

Turn source material into study assets

Generate notes, diagrams, questions and flashcards from a syllabus or an imported PDF.

Measured by · Artefacts created and stored inside the app.

03

Measure understanding continuously

Score every attempt and expose weak topics rather than a single average.

Measured by · A 0–100 weakness score per topic.

04

Make consistency sustainable

Use streaks, XP, quests and small daily goals to reduce drop-off.

Measured by · An active streak and completed daily quests.

Each objective has an observable check — that is exactly what the evidence slide reports.

SCOPE

IN SCOPE

- CSE study loop: DBMS, OS, DSA
- Browser, offline-first
- Individual student
- AI optional, not required

OUT OF SCOPE

- Institutional timetabling
- Grading or proctored exams
- Native mobile application

SUCCESS CRITERIA

47/47 automated tests · production build · live deployment · demo dataset

Five strands inform the design — none closes the loop

The review focused on what each strand does well and what it leaves unresolved for an individual learner.

Strand	What it does well	What it leaves unresolved
Adaptive learning platforms	Sequence content and mastery paths; strong structured curricula	Personal scheduling is weak; the plan rarely reacts to a single student's week
AI tutors & content generation	Explain, summarise and generate questions on demand	Usually cloud-only and disconnected from the learner's own stored data
LMS analytics	Cohort and course dashboards; institutional reporting	An institutional view, not a decision about the next hour of study
Spaced repetition	SM-2 schedules review at the edge of recall	Isolated from the syllabus, exam dates and the rest of the workflow
Gamified learning	XP, streaks and quests sustain habit and return visits	Engagement without adaptation; points do not change what is studied

POSITIONING

LOCAL-FIRST · AI OPTIONAL · CLOSED LOOP

StudyMapper is not another LMS. It is a personal layer that connects planning, content, practice and measurement — and keeps working offline.

Nine modules, arranged as three connected lanes

Capture the material, act on it daily, then reflect — and let reflection change the next action.

CAPTURE & ORGANISE

Subjects & topics — 01

A syllabus tree — subject, unit, topic — with progress on every node.

Notes & imports — 02

Generate notes, or bring PDFs, DOCX and images through the parsing pipeline.

Question banks — 03

Multiple-choice and descriptive questions, with AI-generated model answers.

DECIDE & ACT

Adaptive planner — 04

A day-by-day schedule from difficulty, exam date and panic mode.

Flashcards · SM-2 — 05

Review at the edge of recall; the queue is built from topic and history.

Coding practice — 06

Python problems run in the browser through Pyodide — nothing to install.

REFLECT & SUSTAIN

Analytics & weakness — 07

A 0–100 score per topic, recomputed from the last seven days.

Gamification — 08

XP, levels, streaks, daily quests and achievements for consistency.

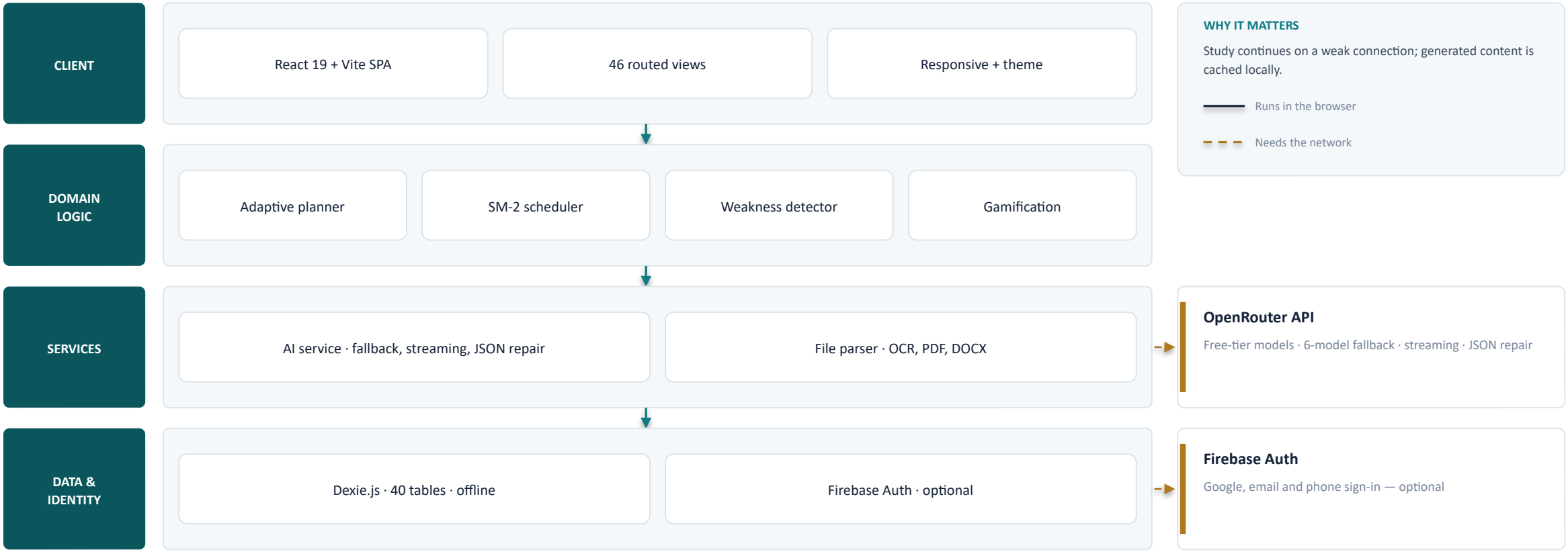
AI assistant — 09

A study chat with six-model fallback, streaming and JSON repair.

Every module writes to the same local store — which is why the third lane can change what the second lane does tomorrow.

A browser-first architecture with optional services

The core study loop keeps working with no server; AI and identity are services, not dependencies.



Privacy by default: attempts and notes stay in the browser; cloud is opt-in.

Built in four phases, each ending in a working slice

The project followed a vertical-slice method: every phase produced something runnable and testable, not a document.

01 DISCOVER

ACTIVITIES

- Peer interviews on study habits
- Audit of existing study tools
- Feature shortlist and constraints

ARTEFACT

Problem statement + feature list

WHY THIS PHASE

Understand the student's week before choosing features.

02 DESIGN

ACTIVITIES

- 40-table data model
- 46-route interface map
- AI prompt and fallback contracts

ARTEFACT

Schema + interface map

WHY THIS PHASE

Decide the data model before writing screens.

03 BUILD

ACTIVITIES

- Vertical feature slices
- Shared component library
- AI service with JSON repair

ARTEFACT

Working modules

WHY THIS PHASE

Keep the application runnable at every step.

04 PROVE

ACTIVITIES

- 47 automated tests
- Production build in 6.21 s
- Seeded demo + 19 screens

ARTEFACT

Verified build and live demo

WHY THIS PHASE

Turn every claim into evidence an examiner can check.

ITERATION & QUALITY

Every slice shipped with tests. AI output is validated by repair-then-retry, not trusted blindly.

A deliberately small stack for a browser-only product

Chosen for a small team, browser-only deployment and an offline-first study loop.

01

Interface

What the student sees and touches.

React 19

Component model for 46 routed views.

Vite

Fast dev loop; 6.21 s production build.

CSS custom properties

One token set drives light and dark themes.

02

Intelligence

Generation and in-browser computation.

OpenRouter

Free-tier models with a six-model fallback chain.

Pyodide

Python runs client-side for coding practice.

Mermaid + KaTeX

Diagrams and mathematical notation, rendered locally.

03

Data

Where the study record actually lives.

Dexie.js + IndexedDB

40-table local store; live queries; offline.

Firebase Auth

Google, email and phone identity — optional.

File APIs

PDF, DOCX and image parsing through Tesseract and pdf.js.

04

Quality

How the build is checked and shown.

Vitest

47 automated tests across the core modules.

Puppeteer

Seeded demo data and genuine interface captures.

Build + PDF

Production bundle and a shareable preview export.

WHAT WE AVOIDED

A custom server fleet · model training or hosting · a paid tier · a separate native build pipeline.

The schema mirrors how a student thinks

Subjects contain topics, topics produce practice, and practice feeds progress — all in one local store.

CURRICULUM 1

4 levels

- Subject
- Unit
- Topic
- Topic content

PRACTICE 2

questions · exams · cards

- Question bank
- Mock exam
- Attempt
- Flashcard · SM-2

PLANNING 3

schedule

- Study plan
- Plan item
- Study session
- Panic mode

PROGRESS 4

signals

- Profile · XP
- Achievement
- Daily quest
- Weakness score

40 tables in total · one Dexie database · no server in the core loop

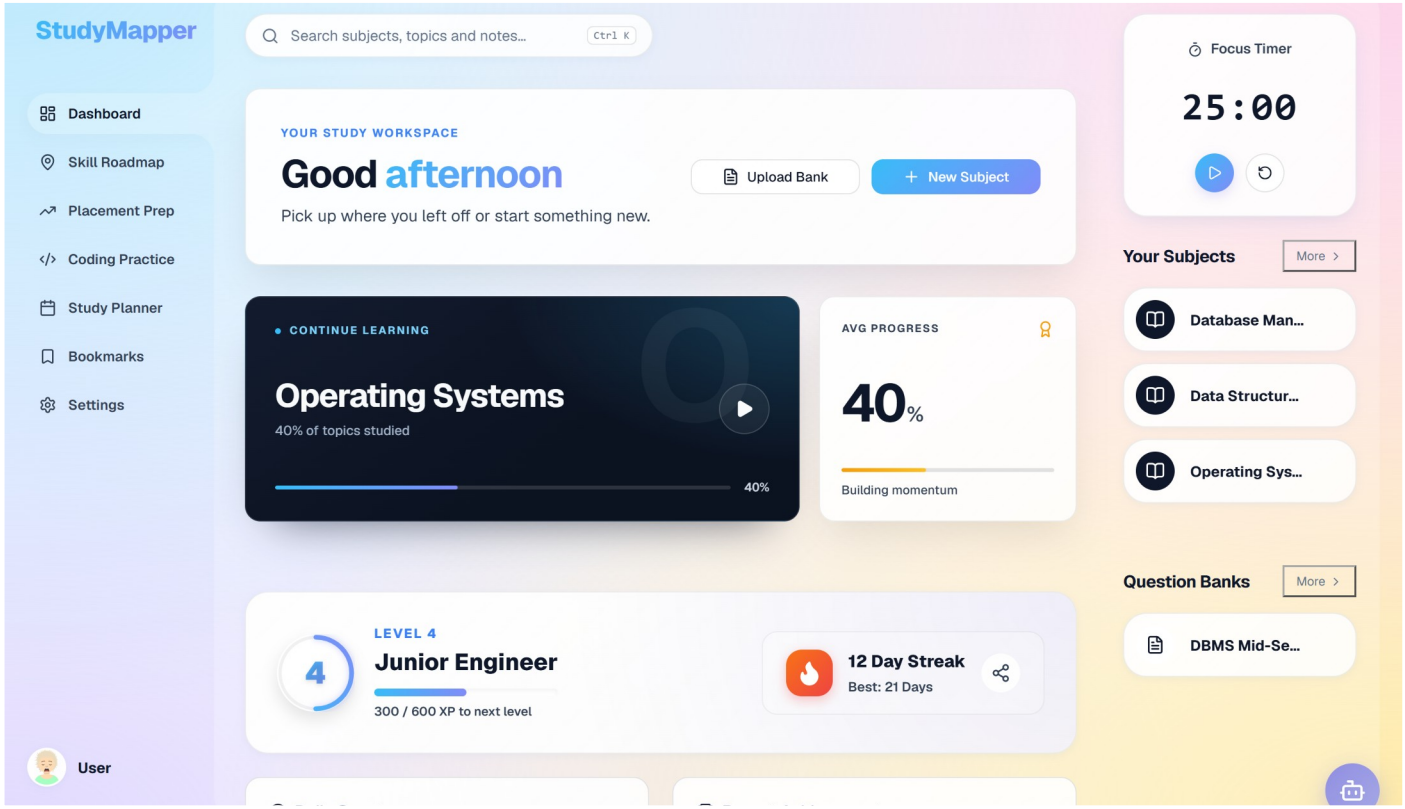
HOW DATA MOVES

1. **Write locally**
   Every action is a Dexie transaction — no network round trip.
2. **React**
   Live queries update the interface immediately, on every screen.
3. **Enrich**
   AI notes and questions are written back into the same tables.
4. **Recompute**
   Recent events update weakness scores and feed the next study plan.

Offline-first: no server is needed to study.

A calm interface built around one question: what next?

Every screen is organised around the next action, not around a dashboard of everything the system knows.



Dashboard at 1440 × 900 — the same build used for the demo, seeded with sample data.

WHAT TO NOTICE

01 One next action

The dashboard opens with “jump back in”, not a wall of metrics.

02 Progress you can feel

Level, XP, streak and daily quests sit beside academic progress.

03 Built for a real evening

Search, focus timer and question banks are one click away.

ALSO IN THE BUILD

- Mock exams
- Coding practice
- Placement prep
- Skill roadmaps
- AI assistant
- Analytics

What is verified — and what is not

Results are reported at the level they were measured: build health and functional completeness.

47/47
automated tests passing
14 test files

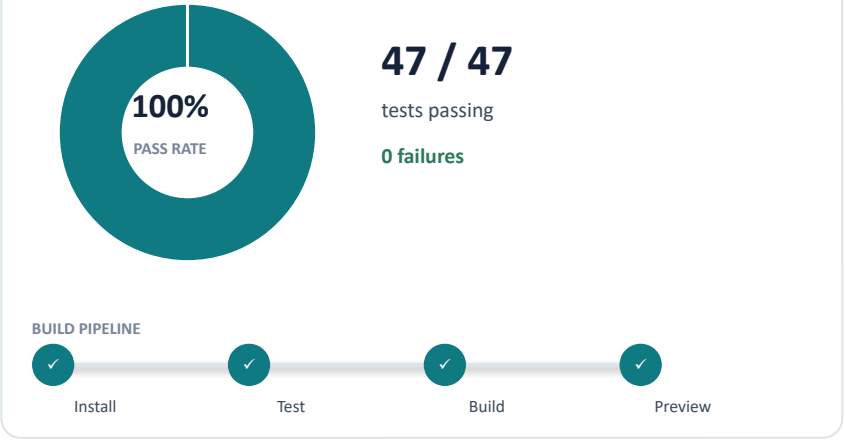
6.21 s
production build
Vite production build

40
database tables
Dexie / IndexedDB

46
routed views
React single-page app

SEEDED DEMO DATASET · 38 topics · 18 note sets · 8 questions · 10-day plan · 12-day streak

Area	Evidence
Automated suite	47 tests across 14 files — all passing; 6.21 s production build
Data layer	40 Dexie tables created, seeded and queried
AI service	Six-model fallback tested; streaming and JSON repair implemented
Interface	19 screens captured from the running application
Deployment	Production build verified; live demo reachable



INTERPRETATION

This evidences functional completeness and build health. It does not yet evidence learning gains — a classroom study is future work.

What it contributes — and what it does not claim

A credible project states its boundaries as clearly as its features.

WHAT IT CONTRIBUTES

A closed loop, not a bundle

Planning, content, practice and measurement share one data model, so evidence actually changes the plan.

AI with a safety net

Six-model fallback, streaming and JSON repair; generated material stays editable, never opaque.

Local-first by design

The core loop runs offline; identity is optional and never required for study.

WHAT IT DOES NOT CLAIM

- no measured learning-outcome study
- no multi-institution deployment
- no teacher or cohort dashboard
- no native mobile application

These are scope decisions, not missing pieces.

DELIBERATE TRADE-OFFS

Decision	Why it was taken	What it costs
Local-first storage	Privacy, instant reads and offline study	No cross-device sync yet
AI-assisted authoring	Cuts content preparation time	Output needs review; free models vary
Breadth over depth	Covers the whole study loop	Each module is intentionally shallow

The goal was a coherent system a student could use end-to-end, not a deep implementation of any single module.

A complete, working learning loop

Not a prototype of one — every part of the loop shown in this deck runs in the deployed build.

ADAPTIVE

Plans recompute from the exam date, topic difficulty and measured weakness.

Regenerate on demand from the latest scores and remaining days.

EXPLAINABLE

Every score and schedule item traces back to stored attempts — no black-box advice.

A topic is flagged weak at 40+; the window is the last seven days.

LOCAL-FIRST

The core loop runs offline; AI and sign-in are optional services.

Notes, plans and attempts live in IndexedDB, not on a server.

EVIDENCE AT A GLANCE

47 / 47	automated tests passing
6.21 s	production build
19	interface captures from the running app

Live demo · study-mapper.vercel.app

Public repository · palturuvenubhaskar/StudyMapper

Built at Anantha Lakshmi Institute of Technology & Sciences (Autonomous), Department of CSE, under the guidance of Ms. C. Sravani.

Evidence first, then scale

The next step is a classroom study; the roadmap follows from what that study reveals.

NOW → 3 MONTHS

Classroom pilot

Pre/post quiz with a real cohort — the first test of learning impact.

Optional sync

Account-based, conflict-safe sync without losing local speed.

Accessibility pass

Keyboard paths, contrast and screen-reader labels.

HOW WE WILL KNOW

SIGNAL · pre/post quiz gain in one cohort

NEXT YEAR

Native mobile shell

Capture small study sessions between classes.

Mentor view

A cohort view for teachers — aggregate, not surveillance.

Content review queue

A workflow for checking AI-generated material before sharing.

HOW WE WILL KNOW

SIGNAL · weekly active students, review turnaround

BEYOND

Multi-language content

Localise notes and questions, not just the interface.

LMS integration

LTI-style hand-off to institutional course spaces.

Longitudinal study

A multi-semester evaluation with an education department.

HOW WE WILL KNOW

SIGNAL · multi-semester retention and outcomes



Sequencing is deliberate: prove that the loop helps a student learn before adding surface area.

Sources used to verify this work

Primary sources first; technical documentation only where it was needed to confirm a claim.

PRIMARY & PLATFORM

1 StudyMapper repository — github.com/palturuvenubhaskar/StudyMapper. Primary source for every implementation claim in this deck.

2 Project abstract — “StudyMapper: AI-Driven Personalized Learning and Academic Management System”, ALITS, 2026–27.

3 React 19 documentation — react.dev. Component model and hooks used across the 46 routed views.

4 Vite documentation — vitejs.dev. Build tooling and the production bundle.

5 Dexie.js documentation — dexie.org. IndexedDB schema, transactions and live queries.

6 Firebase Authentication — firebase.google.com/docs/auth. Google, email and phone sign-in.

LIBRARIES & METHODS

7 OpenRouter API documentation — openrouter.ai/docs. Model routing, streaming and fallback behaviour.

8 Pyodide documentation — pyodide.org. In-browser Python runtime used by coding practice.

9 Tesseract.js — tesseract.js.org; pdf.js — mozilla.github.io/pdf.js; mammoth.js — mammothjs.org. File parsing pipeline.

10 Mermaid — mermaid.js.org; KaTeX — katex.org. Diagram and mathematical rendering.

11 P. A. Woźniak, “SuperMemo 2: Algorithm and implementation”, 1987. Basis of the SM-2 scheduler.

12 Anki manual — apps.ankiweb.net. Reference implementation of spaced-repetition scheduling.

All web sources accessed October 2026. Only sources used to verify the implementation are listed.

Thank you.

Questions & discussion

The deck, the repository and the live build are available for review. Every screenshot in this presentation comes from the running application.

Live demo · study-mapper.vercel.app

GitHub · palturuvenubhaskar

Academic project · 2026–27 · Batch 4

Anantha Lakshmi
INSTITUTE OF TECHNOLOGY & SCIENCES
(AUTONOMOUS)

TEAM

P. Venubhaskar	252G5A0520
T. Vasanth	252G5A0528
V. Adarsh Kumar	252G5A0532
B. Sai Vikas	252G5A0501
C. Narendra	252G5A0502

GUIDE

Ms. C. Sravani

DEPARTMENT

Computer Science & Engineering

INSTITUTE

Anantha Lakshmi Institute of Technology & Sciences (Autonomous)

Ananthapuramu, Andhra Pradesh